T3 Firmware & Bootloader

BACnet change of value · built Nov 13, 2026

BACnet change of value

BACnet Change of Value (COV) Architectural Guide

This document details how the Change of Value (COV) service works in the T3 firmware, including subscription tracking, change detection, state machine execution, and MQTT publishing.

---

1. What is COV and why is it used?

Change of Value (COV) is a BACnet service designed to minimize network congestion and save processing cycles. Instead of a client constantly polling a device for point values (which consumes CPU and network bandwidth), the client subscribes to specific points of interest.

The controller then monitors these points locally and sends a notification to the subscriber **only when a change occurs** (or when the subscription is first registered).

---

2. Subscription Tracking (`COV_Subscriptions[]`)

Subscriptions are tracked in a static array defined in [h_cov.c](file://T3-programmable-controller-on-ESP32/temco_bacnet/src/h_cov.c):

```c
#define MAX_COV_SUBCRIPTIONS 20
static BACNET_COV_SUBSCRIPTION COV_Subscriptions[MAX_COV_SUBCRIPTIONS];
```

Each slot tracks:

- `monitoredObjectIdentifier`: The point type (AI, AO, AV, BI, BO, BV) and its 1-based instance.
- `subscriberProcessIdentifier`: Process ID of the client (usually `1` for MQTT, or client-defined for BACnet).
- `lifetime`: Time remaining in seconds before this subscription expires.
- `dest_index`: Reference to the client's destination address in `COV_Addresses[]`.
- `flag.send_requested`: Set to `true` to request an immediate update.

- `flag.issueConfirmedNotifications`: Indicates whether confirmed or unconfirmed notifications should be sent.

---

3. Subscription Lifetime & Expiration

Subscriptions can have a defined lifetime (in seconds):

- **Infinite Subscriptions**: Registered with `lifetime = 0`. These never expire and remain active until manually unsubscribed or the device reboots.
- **Definite Subscriptions**: Registered with `lifetime > 0`.
- **Expiration Process**:
  1. Every 1 second, the main task calls `handler_cov_timer_seconds(1)`.
  2. For each active subscription with a non-zero lifetime, it decrements the remaining time by `1`.
  3. Once `lifetime` reaches `0`, the subscription is expired (`flag.valid = false`).
  4. The address is cleaned up using `cov_address_remove_unused()`.

---

4. Change Detection & State Machine

COV changes are monitored and processed through two mechanisms:

A. Second-by-Second Timer (`handler_cov_timer_seconds`)

Runs every 1 second inside `Bacnet_Control` and executes local change checks on the registered subscriptions:

- Checks value changes using type-specific functions:
  - `Analog_Value_Change_Of_Value(instance)`
  - `Analog_Input_Change_Of_Value(instance)`
  - `Analog_Output_Change_Of_Value(instance)`
  - `Binary_Value_Change_Of_Value(instance)`
  - `Binary_Input_Change_Of_Value(instance)`
  - `Binary_Output_Change_Of_Value(instance)`
- These compare the current hardware value against a stored backup buffer. If they differ, `cov_send_request` is triggered immediately.

B. State Machine Task (`handler_cov_task`)

Runs every 10ms to manage queueing, clearing, and transmission. It cycles through 5 states:

1. **`COV_STATE_IDLE`**: Resets tracking pointers.

2. **`COV_STATE_MARK`**: Scans the subscriptions. Calls `Device_COV(type, instance)` to see if a point value changed. If yes, sets `flag.send_requested = true`.
3. **`COV_STATE_CLEAR`**: Clears the change flag on the device using `Device_COV_Clear()`.
4. **`COV_STATE_FREE`**: Clean up confirmed transactions that finished or failed.
5. **`COV_STATE_SEND`**: Encodes the property lists (`Device_Encode_Value_List()`) and invokes `cov_send_request()` to transmit the update.

---

5. MQTT and BACnet COV Integration Flow

When MQTT and BACnet are unified, the flow behaves as follows:

MERMAID

```
graph TD
    MQTT[MQTT Client JSON Subscribe] -->|Parse JSON| Mqtt_H[Mqtt_Handler.c]
    Mqtt_H -->|Encode Subscribe APDU| apdu[apdu buffer]
    apdu -->|handler_cov_subscribe| cov_list[Register in COV_Subscriptions with dest = {

    subgraph Detection
        Timer[1s Timer / 10ms Task] -->|Check change| detect[Value changed]
        detect -->|cov_send_request| send[cov_send_request]
    end

    send -->|Update_Value_List| uvl[Update_Value_List in app_main.c]
    uvl -->|#if BACNET_SUB_COV| mqtt_pub[Mqtt_Handler_Send_COV]
    uvl -->|BACnet IP| bacnet_pub[Send unconfirmed broadcast notification]
```

A. Subscribing via MQTT

1. The MQTT client publishes a JSON payload to `temco/cov/tstat11/sub`.
2. `Mqtt_Handler.c` parses the object type, instance, and lifetime.
3. It builds a standard BACnet APDU subscription packet using `cov_subscribe_encode_apdu()`.
4. It calls `handler_cov_subscribe()` directly with a dummy address `src = {0}` (acting as local broadcast).
5. This registers the subscription directly into BACnet's internal `COV_Subscriptions[]` table.

B. Publishing Updates (Unification)

When a point changes (whether subscribed to by a BACnet client or an MQTT client):

1. The BACnet stack detects the change and triggers `cov_send_request()`.

2. `cov_send_request()` calls `Update_Value_List(type, instance)` in `app_main.c`.
3. `Update_Value_List()` reads the value, formats it as a BACnet tag, and:
    - Forwards it to MQTT: Calls `Mqtt_Handler_Send_COV()` (if `BACNET_SUB_COV` is `1`), which publishes the update to the MQTT broker.
    - Forwards it to BACnet: Sends the COV notification packet to the subscriber's address (since the MQTT subscriber used address `{0}`, it sends it as a local broadcast packet, updating all BACnet clients on the network).
4. This ensures that **any** subscription triggers updates to **both** MQTT and BACnet simultaneously.